Authentication and Authorization

What is Authentication?

Authentication answers:

“Who are you?”

It verifies the identity of a user or system.

Examples:

- Username + password
- OTP
- Biometric
- Google login
- API key

If identity is valid → user is authenticated.

Client → Login Request → Server verifies credentials → Server returns token/session

After login, user proves identity using:

- JWT token
- Session cookie
- OAuth access token

1️⃣ Session-Based Authentication (Traditional)

Flow:

1. User logs in.
2. Server creates session in memory/database.
3. Server sends session ID in cookie.
4. Client sends cookie with every request.

Client → Cookie (sessionId) → Server → Session Store

Pros:

- Simple
- Easy to revoke

Cons:

- Not stateless
- Harder to scale (needs shared session store)

2️⃣ JWT (Token-Based Authentication)

JWT = JSON Web Token

Server does NOT store session.

Instead:

1. User logs in.
2. Server creates signed token.
3. Client stores token (usually in HTTP-only cookie).
4. Client sends token in Authorization header.

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Server verifies signature each request.

JWT Structure

Header.Payload.Signature

Example payload:

```
{
 "userId": "123",
 "role": "admin",
 "exp": 1710000000
}
```

Pros:

- Stateless
- Scalable

- Good for microservices

Cons:

- Harder to revoke
- Must manage expiration carefully

Most modern SaaS apps use JWT.

3️⃣ OAuth 2.0 (Third-Party Login)

Used for:

- Login with Google
- Login with GitHub

Flow:

Client → Google → Google verifies → Returns token → Server validates token

OAuth is authorization delegation protocol.

Often combined with JWT in backend.

2️⃣ Authorization (AuthZ)

Authentication = Who are you?
Authorization = What are you allowed to do?

Even if user is logged in:

- Can they delete users?
- Can they view admin dashboard?
- Can they access another user's data?

That’s authorization.

Difference Between Authentication and Authorization

Authentication	Authorization
Verifies identity	Checks permissions
Happens first	Happens after authentication
Example: login	Example: access admin route

4️⃣ Authorization Models

1️⃣ RBAC (Role-Based Access Control)

Users have roles:

- Admin
- User
- Manager

Example:

```
if (user.role !== "admin") {
  return res.status(403).json({ message: "Forbidden" });
}
```

Simple and widely used.

2️⃣ Permission-Based (More Granular)

Instead of roles, assign permissions:

- create_user
- delete_user
- view_reports

More flexible than RBAC.

Often:

Role → has many permissions
User → has role

3️⃣ ABAC (Attribute-Based Access Control)

Based on attributes like:

- User role
- Time
- Location
- Resource owner

Example:

User can edit only their own post.

```
if (post.userId !== currentUser.id) {
  return res.status(403).json({ message: "Forbidden" });
}
```

More dynamic and powerful.

API Gateway + Authentication (Microservices)

In large systems:

Client → API Gateway → Microservices

Gateway:

- Verifies JWT
- Extracts user info
- Forwards request with user context

Microservices trust gateway.

This avoids duplicate auth logic everywhere.

Companies like:

- Netflix
- Amazon

Use centralized authentication at gateway level.

Security Best Practices

✅ **Always use HTTPS**

✅ **Store JWT in HTTP-only cookies (avoid XSS)**

✅ **Use short-lived access tokens**

✅ **Use refresh tokens**

✅ **Hash passwords using bcrypt**

✅ **Never store plain passwords**

✅ **Validate input**

✅ **Implement rate limiting**